EXP NO: 4	Implementation Of A Convolutional Neural Network (Cnn)

AIM:

To implement a Convolutional Neural Network (CNN) for image classification on the CIFAR-10 dataset, evaluate its test accuracy, and visualize the learned first-layer filters and sample predictions.

ALGORITHM:

1. Load the CIFAR-10 training and test sets, normalize the images and create DataLoaders with batch size 64.
2. Define SimpleCNN: Conv2d(3, 16, 3x3) - ReLU - MaxPool, Conv2d(16, 32, 3x3) - ReLU - MaxPool, flatten (32 x 8 x 8), Linear(2048, 128) - ReLU, Linear(128, 10).
3. Use the cross-entropy loss and the Adam optimizer (learning rate 0.001).
4. Train the network for 5 epochs and print the average loss of each epoch.
5. Evaluate the trained model on the test set and compute the test accuracy.
6. Visualize the first eight filters of the first convolutional layer and display eight test images with their actual and predicted classes.

CODE:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

print("PyTorch version:", torch.__version__)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5),
        (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

trainloader = torch.utils.data.DataLoader(
    trainset,
    batch_size=64,
    shuffle=True
)

testset = torchvision.datasets.CIFAR10(
    root="./data",
    train=False,
    download=True,
    transform=transform
)
```

```
testloader = torch.utils.data.DataLoader(
    testset,
    batch_size=64,
    shuffle=False
)

classes = trainset.classes

print("Classes:", classes)

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()

        self.conv1 = nn.Conv2d(
            in_channels=3,
            out_channels=16,
            kernel_size=3,
            padding=1
        )

        self.pool = nn.MaxPool2d(2, 2)

        self.conv2 = nn.Conv2d(
            in_channels=16,
            out_channels=32,
            kernel_size=3,
            padding=1
        )

        self.fc1 = nn.Linear(32 * 8 * 8, 128)

        self.fc2 = nn.Linear(128, 10)

        self.relu = nn.ReLU()

    def forward(self, x):

        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)

        x = self.conv2(x)
        x = self.relu(x)
        x = self.pool(x)

        x = x.view(x.size(0), 32 * 8 * 8)

        x = self.fc1(x)
        x = self.relu(x)

        x = self.fc2(x)

        return x

model = SimpleCNN().to(device)

loss_function = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
```

```
)

print(model)

epochs = 5

for epoch in range(epochs):

    model.train()

    running_loss = 0.0

    for inputs, labels in trainloader:

        inputs = inputs.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(inputs)

        loss = loss_function(outputs, labels)

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

    average_loss = running_loss / len(trainloader)

    print(
        f"Epoch {epoch + 1}/{epochs}, "
        f"Loss: {average_loss:.4f}"
    )

model.eval()

correct = 0
total = 0

with torch.no_grad():

    for images, labels in testloader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total

i    (f"\                  {            f}%")
print(f"\nTest Accuracy: {accuracy:.2f}%")

def visualize_filters(layer, n_filters=8):
```

```
    filters = layer.weight.data.clone().cpu()

    fig, axes = plt.subplots(
        1,
        n_filters,
        figsize=(15, 4)
    )

    for i in range(n_filters):

        f = filters[i]

        f = (
            f - f.min()
        ) / (
            f.max() - f.min() + 1e-8
        )

        axes[i].imshow(
            f.permute(1, 2, 0)
        )

        axes[i].axis("off")

        axes[i].set_title(
            f"Filter {i + 1}"
        )

    plt.tight_layout()

    plt.show()

visualize_filters(model.conv1)

dataiter = iter(testloader)

images, labels = next(dataiter)

images = images.to(device)

with torch.no_grad():

    outputs = model(images)

    _, predicted = torch.max(outputs, 1)

plt.figure(figsize=(12, 6))

for i in range(8):

    image = images[i].cpu()

    image = image * 0.5 + 0.5

    image = image.permute(1, 2, 0)

    plt.subplot(2, 4, i + 1)

    plt.imshow(image)

    plt.title(
        f"Actual: {classes[labels[i]]}\n"
        f"Predicted: {classes[predicted[i]]}"
```

```
    )

    plt.axis("off")

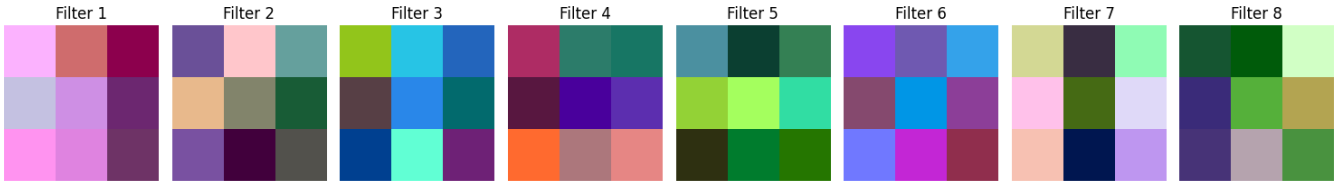
plt.tight_layout()

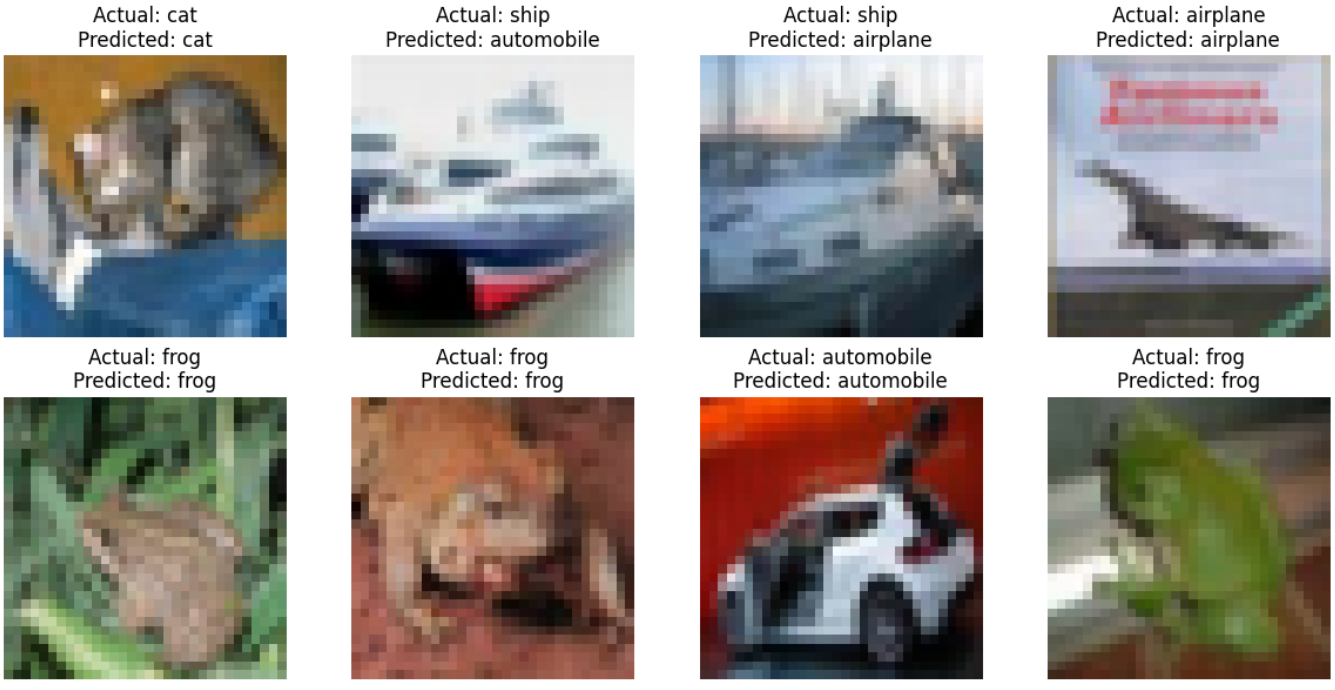
plt.show()

print("\nExperiment 4 completed successfully.")
```

OUTPUT:

```
PyTorch version: 2.11.0+cpu
Using device: cpu
100%|██████████| 170M/170M [04:24<00:00, 645kB/s]
Classes: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
SimpleCNN(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (fc1): Linear(in_features=2048, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
  (relu): ReLU()
)
Epoch 1/5, Loss: 1.4612
Epoch 2/5, Loss: 1.0927
Epoch 3/5, Loss: 0.9427
Epoch 4/5, Loss: 0.8392
Epoch 5/5, Loss: 0.7655
Test Accuracy: 68.61%
Experiment 4 completed successfully.
```





RESULT:

The CNN was implemented and trained on CIFAR-10 for 5 epochs. The training loss decreased from 1.4612 to 0.7655 and the model achieved a test accuracy of 68.61%.